WideResNet-28-4 + SE Attention

Custom CNN model cho CIFAR-10 / CIFAR-100 | ~5.9 triệu parameters

📋 Bảng ký hiệu

- **Conv** = Convolution — bộ lọc quét ảnh, trích xuất đặc trưng
- **BN** = BatchNorm — chuẩn hoá dữ liệu cho đều, giúp train nhanh hơn
- **ReLU** = Hàm kích hoạt — giữ số dương, bỏ số âm
- **Dropout** = Random tắt 30% neuron — chống học vẹt (overfitting)
- **SE** = Squeeze-Excitation — chấm điểm channel nào quan trọng
- **Sigmoid** = Ép giá trị về 0~1 — dùng làm điểm quan trọng
- **AvgPool** = Lấy trung bình — nén vùng pixel thành 1 số
- **FC** = Fully Connected — kết nối toàn bộ → ra kết quả cuối
- **(+)** = Cộng input gốc vào output (skip connection)
- **× Input** = Nhân điểm quan trọng vào từng channel

S = Stride (bước nhảy) — S=1: giữ nguyên kích thước, S=2: thu nhỏ ÷2

P = Padding (viền thêm) — thêm pixel 0 quanh ảnh để giữ kích thước

C = Channels (số lớp đặc trưng) — RGB có 3, qua conv tạo thêm nhiều lớp

3×3 = Kích thước bộ lọc — quét vùng 3×3 pixel mỗi lần

1️⃣ Tổng quan — Ảnh đi qua những gì

Tại sao cần 3 Group?

Giống mắt người nhìn ảnh:

Group 1 (dí sát): thấy nét nhỏ — cạnh, góc, sọc, màu

Group 2 (lùi ra): ghép nét nhỏ → hình dạng — tai, mắt, cánh, bánh xe

Group 3 (nhìn xa): ghép hình dạng → nhận ra vật — "đây là con mèo"

Cần cả 3 bước. Thiếu bước nào → đoán sai.

🖼️ **INPUT**
32×32×3
ảnh màu RGB

↓

Conv 3×3 S=1, P=1 *bộ lọc đầu tiên* → **32×32×16** 16 lớp đặc trưng sơ bộ

↓

GROUP 1 — nhìn chi tiết nhỏ

📥 **Đầu vào:** 32×32×16 → 📤 **Đầu ra:** 32×32×64

🔍 **Stride = 1:** kích thước ảnh giữ nguyên 32×32

📊 **Channels: 16 → 64:** tạo thêm nhiều lớp đặc trưng (×4 widen factor)

🔁 **Lặp 4 lần:** mỗi lần lọc thêm 1 lớp chi tiết

🧱 **WideResBlock — cấu trúc 1 block**

Input w×h×C *dữ liệu vào* → **BN** *chuẩn hoá* → **ReLU** *bỏ số âm* → **Conv 3×3** S=1, P=1 *lọc đặc trưng* → **Dropout** p=0.3 *tắt 30% ngẫu nhiên*

↓

BN *chuẩn hoá* → **ReLU** *bỏ số âm* → **Conv 3×3** S=1, P=1 *lọc lần 2* → **SE Block** *chấm điểm channel* → (+) → **Output** w×h×C' *kết quả*

↗️ **Skip connection (đường tắt):** Input gốc được **copy nguyên** rồi **cộng thẳng** vào Output tại dấu (+)
Tác dụng: dù conv xử lý sai → vẫn giữ được thông tin gốc → gradient không bị mất khi train

⟲ **Block trên lặp lại 4 lần liên tiếp trong Group 1**

🔎 **SE Block bên trong mỗi Block — chấm điểm channel quan trọng**

Input w×h×C → **AvgPool** w×h → 1×1 *nén mỗi lớp thành 1 số* → **FC** C → C/16 *nén xuống tóm tắt* → **ReLU** *bỏ số âm* → **FC** C/16 → C *giãn lại đủ C điểm* → **Sigmoid** *ép về 0~1* → **× Input** *nhân điểm vào channel*

Ví dụ: ảnh con mèo → channel "mắt tròn" được điểm 0.9 (giữ nguyên) → channel "nền tường" được điểm 0.1 (mờ đi)

↓ output: 32×32×64

GROUP 2 — nhìn hình dạng

📥 **Đầu vào:** 32×32×64 → 📤 **Đầu ra:** 16×16×128

🔍 **Block đầu stride = 2:** ảnh thu nhỏ ÷2 (32→16) — "lùi ra xa để thấy hình dạng"

📊 **Channels: 64 → 128:** hình dạng phức tạp hơn nét vẽ → cần nhiều lớp ghi chú hơn

🧱 **WideResBlock ×4 — cấu trúc giống hệt Group 1**

Input → **BN** → **ReLU** → **Conv 3×3** block 1: S=2, block 2-4: S=1 → **Dropout** → **BN** → **ReLU** → **Conv 3×3** → **SE** → (+) → **Output**

↗️ Skip connection + SE Block bên trong — **giống hệt Group 1**

⚠️ **Khác Group 1 ở đâu?** Chỉ khác block đầu tiên dùng **stride=2** (thu nhỏ ảnh ÷2). Shortcut phải dùng Conv 1×1 (S=2) để kích thước khớp nhau khi cộng (+). Ba block còn lại stride=1 bình thường.

↓ output: 16×16×128

GROUP 3 — nhận diện tổng thể

📥 **Đầu vào:** 16×16×128 → 📤 **Đầu ra:** 8×8×256

🔍 **Block đầu stride = 2:** ảnh thu nhỏ ÷2 (16→8) — "nhìn toàn cảnh"

📊 **Channels: 128 → 256:** nhận diện mức trừu tượng → cần nhiều lớp nhất

🧱 **WideResBlock ×4 — cấu trúc giống hệt Group 1 & 2**

Input → **BN** → **ReLU** → **Conv 3×3** block 1: S=2, block 2-4: S=1 → **Dropout** → **BN** → **ReLU** → **Conv 3×3** → **SE** → (+) → **Output**

↗️ Skip connection + SE Block bên trong — **giống hệt Group 1 & 2**

⚠️ **Khác Group 2 ở đâu?** Chỉ khác số channel (128→256) và kích thước ảnh (16→8). Cấu trúc block **100% giống**.

↓ output: 8×8×256

🎯 Phần cuối — đưa ra kết luận

BN *chuẩn hoá lần cuối* → **ReLU** *bỏ số âm* → **AvgPool** 8×8 → 1×1 *nén 256 lớp thành 256 số* → **Flatten** *xếp thành 1 hàng dài* → **FC** 256 → 10 *256 số → 10 xác suất* → 🏆 **10 classes** (100 cho CIFAR-100) *class cao nhất = kết quả*

📊 Bảng tổng kết

Thành phần	Input size	Output size	Vai trò
Stem Conv	32×32×3	32×32×16	Chuyển ảnh RGB → dạng model đọc được
Group 1 (×4 block)	32×32×16	32×32×64	Nhìn chi tiết nhỏ: cạnh, góc, sọc, màu
Group 2 (×4 block)	32×32×64	16×16×128	Ghép chi tiết → hình dạng: tai, mắt, cánh
Group 3 (×4 block)	16×16×128	8×8×256	Ghép hình dạng → nhận ra vật thể
AvgPool + FC	8×8×256	10	Tổng hợp → ra 10 xác suất → chọn class cao nhất

Kiến trúc:	WideResNet-28-4 + SE Attention
Depth (độ sâu):	28 layers = 1 stem + 3 groups × 4 blocks × 2 conv/block + 1 FC
Widen factor:	4 → channels nhân rộng gấp 4 so với ResNet gốc
Tổng blocks:	3 groups × 4 blocks = 12 blocks
Parameters:	~5.9 triệu con số model phải học
Optimizer:	SGD (lr=0.1, momentum=0.9, weight_decay=5e-4, nesterov=True)
Scheduler:	CosineAnnealingLR — learning rate giảm dần theo hình cos
Augmentation:	RandomCrop + HFlip + AutoAugment + Cutout — biến đổi ảnh ngẫu nhiên khi train
Label smoothing:	0.1 — không cho model quá tự tin vào 1 đáp án
Dropout:	0.3 — random tắt 30% neuron mỗi lần train